

Password Cracking and Hash Security

NECKER

November 18, 2025

Contents

1	Why Password Cracking	2
1.1	The Hash Function	2
2	John the Ripper (JtR): Cracking Tool	2
2.1	John the Ripper Cracking Modes	2
3	Precomputed Attacks and Defense: Rainbow Tables and Salting	3
3.1	Rainbow Tables	3
3.2	The Countermeasure: Salting	3
4	Password Management on Linux Systems	4
4.1	Salted Hash Format in /etc/shadow	4
4.1.1	Example of an /etc/shadow Line (SHA-512)	5

1 Why Password Cracking

Password cracking is a fundamental security analysis technique, utilized both by attackers to compromise systems and by security professionals to test the robustness of their defenses.

- **Leaked Passwords:** Stolen credentials are often *easily* obtainable online following **Data Breaches** or **misconfigured servers**. Services like <https://haveibeenpwned.com/> allow verifying whether one's own credential has been compromised.
- **Data Format:** A stolen password is not usually stored in cleartext, but in its encrypted (or pseudo-encrypted) form: the **Hash**.

1.1 The Hash Function

A **hash function** is a mathematical algorithm that takes an input (the password) of variable length and produces an output (the hash) of **fixed length**.

$$\text{Hash} = H(\text{Password})$$

Key characteristics:

- **One-Way:** It is easy to compute the hash from the password, but extremely difficult to trace back to the password starting from the hash (pre-image resistance property).
- **Deterministic:** The same password always produces the exact same hash (unless a Salt is utilized).

Hypothetical example (with SHA-256):

$$\text{matteo} \xrightarrow{H} \text{c612a8d03...}$$

2 John the Ripper (JtR): Cracking Tool

John the Ripper (JtR) is a highly popular free and open-source software for password cracking.

- **Integrated Functionality:** It combines a series of crackers into a single package.
- **Autodetect:** It *autodetects* the hash type of the password (although it is always recommended to specify it, for example using the built-in Kali Linux tool: **hash-identifier** and then defining it in **JtR** with the **-format** parameter).
- **Versatility:** It can be executed against various formats of encrypted and salted passwords (such as those of Linux, Windows, databases, etc.).

2.1 John the Ripper Cracking Modes

- **Wordlist (Dictionary):** The simplest and most effective mode. John attempts to calculate the hash of each word specified in a dictionary file (a text file containing one word per line) and checks for a match.
- **Single:** It utilizes user information (username, full name, etc.) and applies **mangling** rules (common modifications like character substitutions, added numbers) as password candidates.

- **Incremental/Bruteforce:** The most powerful mode; it attempts all possible combinations of characters. It is not guaranteed to ever terminate (theoretically infinite computation time for sufficiently long passwords).
- **External:** Allows the use of custom rules and algorithms to generate passwords to be tested.

3 Precomputed Attacks and Defense: Rainbow Tables and Salting

3.1 Rainbow Tables

Rainbow Tables are pre-computed databases of hashes. To mitigate attacks based on **unsalted** passwords:

- Passwords and random strings are processed with the hash function, and the mapping {input / hash} is stored.
- The attacker performs a direct **reverse lookup** on the stolen hash, skipping the computation phase.
- **Advantage:** Extreme speed.
- **Disadvantage:** They require an immense amount of storage space.

3.2 The Countermeasure: Salting

To nullify the effectiveness of Rainbow Tables, **Salt** is utilized.

- **Definition:** The Salt is a **random** value (of fixed length and cryptographically strong) that is **added** to the password before the hash computation.
- **Process:** $\text{Hash} = H(\text{Password} + \text{Salt})$. Both the Hash and the Salt are stored.
- **Effectiveness:** Each password (even identical ones) with a different Salt produces a completely different Hash. This makes pre-computing Rainbow Tables impracticable, as a separate table would be required for every possible Salt.

4 Password Management on Linux Systems

Linux systems separate user information from encrypted passwords for security reasons.

- **/etc/passwd:** A world-readable text file that contains basic user information. In modern versions, the password is replaced with a placeholder (e.g., `x`) for security purposes.
- **/etc/shadow:** A text file readable **only by the root user** that contains password hashes and security data (expiration, etc.).

4.1 Salted Hash Format in /etc/shadow

Field	Name	Data Type	Description
1	Username	String	The account identifier (e.g., <code>madam</code>).
2	Password Hash	Complex string	Contains the password hash. The format is <code>\$id\$salt\$hash</code> , where <code>\$id\$</code> indicates the algorithm (e.g., <code>\$6\$</code> for SHA-512).
3	Last Change	Integer	Date when the password was last changed (in days elapsed since January 1, 1970, the Unix epoch).
4	Minimum Days	Integer	Minimum number of days that must elapse before the user can change the password again.
5	Maximum Days	Integer	Maximum number of days after which the password must be changed.
6	Warning Days	Integer	Number of days before password expiration during which the user will receive a warning.
7	Inactive Days	Integer	Number of days after account expiration before the account becomes disabled.
8	Expiration Date	Integer	Date (in days from the Unix epoch) when the entire account permanently expires.
9	Reserved	String	Field reserved for future use (often empty).

The hash format in `/etc/shadow` encodes the hash type, Salt, and final Hash:

```
username: $id$salt$hashed_password: ...
```

- **\$id** Indicates the hashing algorithm utilized. For example:
 - `$1$` = MD5
 - `$5$` = SHA-256
 - `$6$` = SHA-512 (Modern standard)
- **salt:** The value of the Salt utilized, stored in cleartext.
- **hashed_password:** The final result of the hash.

4.1.1 Example of an `/etc/shadow` Line (SHA-512)

```
madam:$6$reRr06Pic/lkDt1E$6B8znS3dxYmn..  
..95oeJmWxjlU.../1a.:1003:1003:Adam Miller:/home/madam:/bin/sh
```

In this example:

- **User:** madam
- **Algorithm:** \$6\$ (SHA-512)
- **Salt:** reRr06Pic/lkDt1E
- **Encrypted Hash:** 6B8znS3dxYmn95oeJmWxjlU...